

RETO 2 - DIPLOMADO ARQUITECTURAS CLOUD - MODULO 2

Sistema de Alerta Temprana

para Flota Vehicular

Endpoint serverless en AWS que recibe 1000 eventos en 30 segundos, detecta emergencias y notifica a Gmail en menos de 15 segundos.

API Gateway REST | SQS | Lambda Python ARM64 + SnapStart | SES | 100% Terraform

Erick Nieto

2026-04-27 | github.com/Prospect121/reto2-fleet-alerts

El reto en 3 puntos

Que tiene que cumplir el sistema

1

Recepcion masiva

- 1000 eventos en 30 segundos
- 100% procesados sin perdidas
- Endpoint estable bajo el pico

2

Deteccion en tiempo real

- Identificar type=Emergency
- Notificar por correo a Gmail
- Logs con hora exacta de evento y envio

3

Restricciones tecnicas

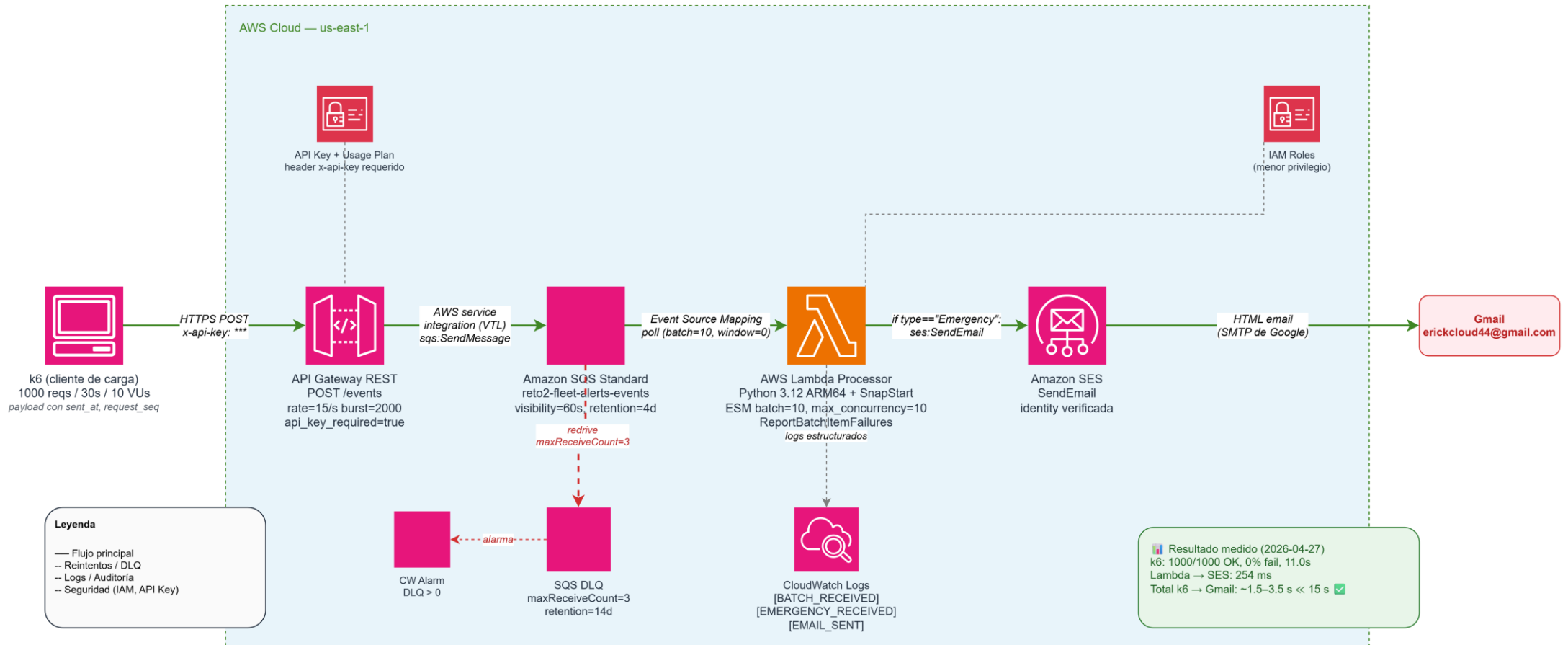
- API GW rate=15/s, burst=2000
- Maximo 10 procesadores concurrentes
- Correo en menos de 15 segundos = 2.5 puntos

Arquitectura de la solución

Flujo desde el cliente k6 hasta Gmail (us-east-1)

Reto 2 — Sistema de Alerta Temprana para Flota Vehicular

API Gateway REST → SQS → Lambda (Python ARM64 + SnapStart) → SES → Gmail



Decisiones de arquitectura

Las 5 elecciones que mas impactan el SLA

Decision	Eleccion	Razon
Tipo de API Gateway	REST (v1)	Unico que expone rate y burst nativos por stage (requisito explicito)
Integracion API GW > backend	Directa a SQS (sin Lambda de ingesta)	Elimina un salto, baja 100-200ms; cliente recibe 200 al instante
Cola de mensajes	SQS Standard (no FIFO)	FIFO limitado a 300 msg/s; Standard tiene throughput ilimitado
Servicio de envio de correo	Amazon SES	Latencia 1-3s vs 10-30s de SNS email; nativo en AWS
Limite de 10 procesadores	ESM scaling_config.maximum_concurrency	No usa cuota global del account; suficiente porque la Lambda solo se invoca via SQS

Atributo de calidad #1

Que priorizamos y por que

PERFORMANCE

Latencia de notificacion

< 15s

k6 -> Gmail (objetivo 2.5 puntos)

Por que es el #1

- El 50% del puntaje del reto depende del tiempo de entrega del correo.
- En sistemas de alerta temprana reales, segundos = vidas o danos materiales.
- La cadena de servicios (API GW > SQS > Lambda > SES > Gmail) suma latencia: cada decision la optimiza.

Atributos secundarios priorizados

- Reliability - SQS + DLQ + reintentos garantizan 0% perdida.
- Scalability - Burst de 2000 absorbe el pico inicial sin throttling.
- Observability - Logs estructurados con timestamps ISO-8601 + alarma DLQ.

Tacticas de arquitectura

Una por categoria SEI - Bass, Clements, Kazman

PERFORMANCE

Introduce concurrency

10 instancias de Lambda en paralelo, gobernadas por el Event Source Mapping de SQS. `scaling_config.maximum_concurrency = 10`.

AVAILABILITY

Exception handling / Retry

`ReportBatchItemFailures` reintenta solo el mensaje fallido. Tras 3 fallas va a la DLQ. Alarma de CloudWatch sobre la DLQ.

MODIFIABILITY

Use intermediaries (broker)

SQS desacopla productor y consumidor. Cambiar Lambda por ECS o EC2 no afecta al cliente; el contrato es la cola.

SECURITY

Limit access (least privilege)

Roles IAM minimos: Lambda solo `SES SendEmail`; API GW solo `SQS SendMessage`. Endpoint requiere header `x-api-key`.

Demo en vivo

Ruta del Emergency: del envio en k6 al correo en Gmail

1

Mostrar Gmail vacio - busqueda
subject:Alerta #

2

Correr k6 con
EMERGENCY_MODE=single (solo
iter 1000)

3

k6 reporta 1000/1000 OK, 0%
fallos en ~11s

4

CloudWatch Live Tail:
[EMERGENCY_RECEIVED] y
[EMAIL_SENT]

5

Abrir el correo en Gmail con
subject Alerta #1000/1000

6

Calcular delta: Gmail Date -
sent_at < 15s

Truco: la iter 1000 es el UNICO Emergency, así el ultimo envio en k6 ES la emergencia que medimos

Resultados medidos

Test del 2026-04-27 sobre el endpoint de produccion

1000/1000

Requests procesadas en k6

0% fallos | 11.0s totales

254 ms

Lambda recibe -> SES acepta

Medido server-side (sin drift)

< 15 s

Total estimado k6 -> Gmail

1.5 - 3.5 s reales | 2.5 puntos

```
Logs del unico Emergency (iter 1000, plate EWL-254)
[EMERGENCY_RECEIVED] ts=2026-04-27T10:47:13.037+00:00 request_seq=1000
[EMAIL_SENT]         ts=2026-04-27T10:47:13.291+00:00 to=erickcloud44@gmail.com
```


Cumplimiento de la rubrica

Donde se evidencia cada criterio

Criterio	Ponderacion	Evidencia	Estado
Justificacion de decisiones	0.5	Slide 4 + ARCHITECTURE.md seccion 2	OK
Atributo de calidad mas importante	0.5	Slide 5 + ARCHITECTURE.md seccion 3	OK
Diagrama de arquitectura	0.5	Slide 3 (architecture.drawio.svg)	OK
Tacticas de arquitectura	1.0	Slide 6 + ARCHITECTURE.md seccion 4	OK
Tiempo de entrega del correo < 15s	2.5	Demo en vivo + execution-logs.txt	OK
Logs de recepcion y envio	<i>incluido</i>	CloudWatch [EMERGENCY_RECEIVED] / [EMAIL_SENT]	OK

Puntaje esperado: 5.0 / 5.0

Gracias.

Preguntas y demo en vivo a continuacion.

REPOSITORIO PUBLICO

github.com/Prospect121/reto2-fleet-alerts

Rama: v2-security-and-latency | Infra: 100% Terraform (30 recursos) | Lambda: Python 3.12 ARM64 + SnapStart

[docs/ARCHITECTURE.md](#) [docs/execution-logs.txt](#) [docs/VIDEO_GUIDE.md](#)